

Informe de Proyecto: Optimización de Particionamiento de Sistemas

4. Entregables del Proyecto

4.2 Documentación Formal del Sistema

El sistema se ha diseñado bajo un enfoque modular para garantizar escalabilidad y facilidad de mantenimiento. Se divide en tres componentes fundamentales:

Módulo de Persistencia y Validación (core/loader.py): Gestiona la entrada de datos desde archivos CSV, transformándolos en estructuras matriciales de NumPy. Valida requisitos críticos como la simetría de la matriz y la ausencia de pesos negativos.

Núcleo de Procesamiento Algorítmico (core/engine.py): Centraliza la lógica de decisión. Implementa un despachador que selecciona el algoritmo óptimo (Exhaustivo vs. Heurístico) basándose en la complejidad computacional del grafo de entrada ($N=12$).

Interfaz de Usuario (gui/app.py): Proporciona un entorno interactivo basado en Tkinter que facilita tanto la carga masiva de datos como la experimentación visual mediante un lienzo de dibujo dinámico.

Fundamentación de Algoritmos:

Exploración Exhaustiva (Backtracking): Implementado mediante una pila (LIFO) para evitar el desbordamiento de la pila de llamadas del sistema. Este método explora el espacio de estados completo para garantizar que el "Valor de Pérdida" sea el mínimo absoluto posible.

Algoritmo Greedy (Heurística): Utilizado para grafos donde $N \geq 12$. Su lógica se basa en la minimización del costo local, asignando componentes a particiones de manera secuencial para ofrecer una respuesta inmediata incluso en redes complejas.

4.3 Manual Técnico

Especificaciones del Entorno:

Lenguaje: Python 3.10+.

Dependencias Críticas: NetworkX (grafos), Matplotlib (renderizado), NumPy & Pandas (matrices), Tkinter (GUI).

Diccionario de Funciones Clave:

calcular_costo_corte(matriz, particion): Función centralizada en metrics.py que calcula el costo total de las dependencias externas (aristas que cruzan fronteras de grupo).

ejecutar(modos): Método en la GUI que integra la lógica del Canvas y la carga de archivos, asegurando que la matriz de adyacencia generada sea válida antes de pasar al motor de resolución.

visualizar_graficamente(particion, costo): Genera un layout tipo spring para el grafo, asignando una paleta de colores diferenciada para identificar visualmente los grupos resultantes.

4.4 Manual de Usuario

Procedimiento de Operación:

Preparación de Datos (CSV): Asegúrese de que su archivo tenga etiquetas en la primera fila y columna. El sistema detectará automáticamente los nombres de los servicios (ej. Auth, Users, Cart).

Uso de la Interfaz:

Pestaña Cargar CSV: Use el botón "Cargar CSV" para importar archivos. El sistema confirmará la carga exitosa mostrando el nombre del archivo debajo del botón.

Pestaña Dibujar Grafo: Cree nodos con Click Izquierdo y conéctelos con Click Derecho (primero origen, luego destino) ingresando el peso.

Configuración de Particiones: Ingrese el valor de k (número de grupos deseados) en el campo correspondiente antes de ejecutar.

Análisis de Salida: Revise el "Valor de Pérdida" en la consola y analice la distribución espacial en el gráfico generado.

4.5 Resultados Experimentales

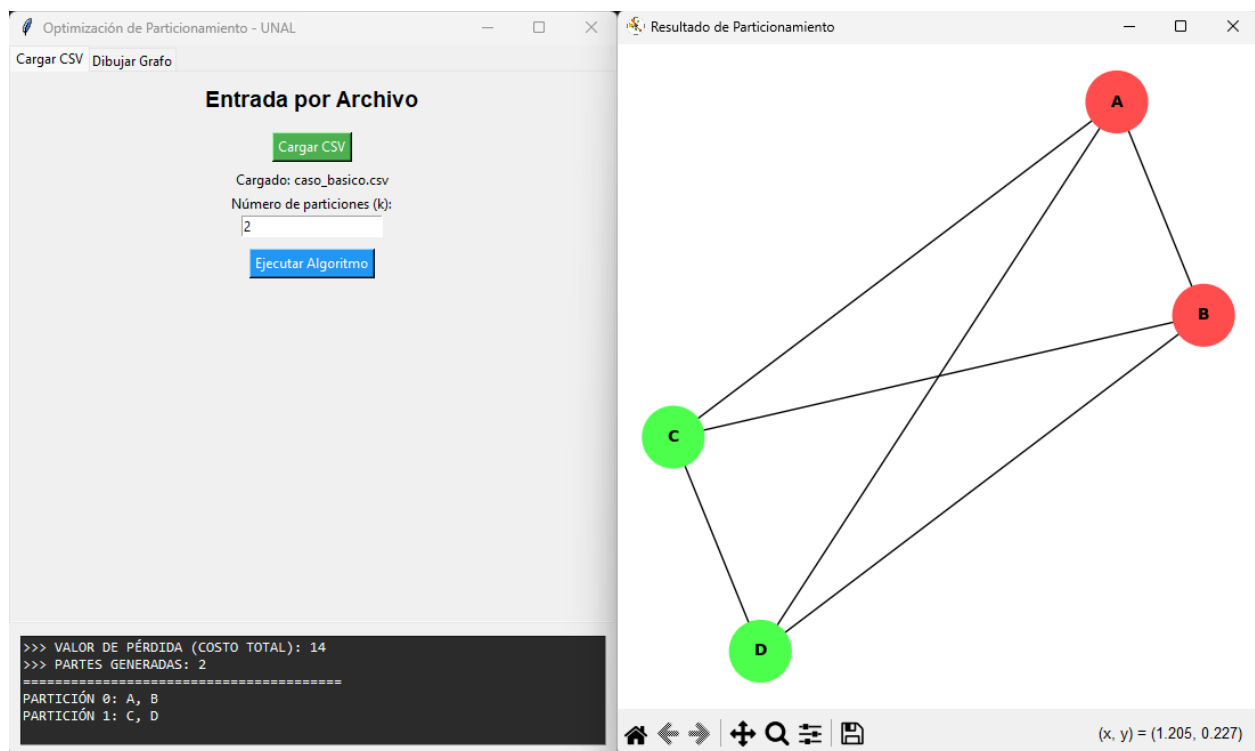


Foto 1. Caso Basico

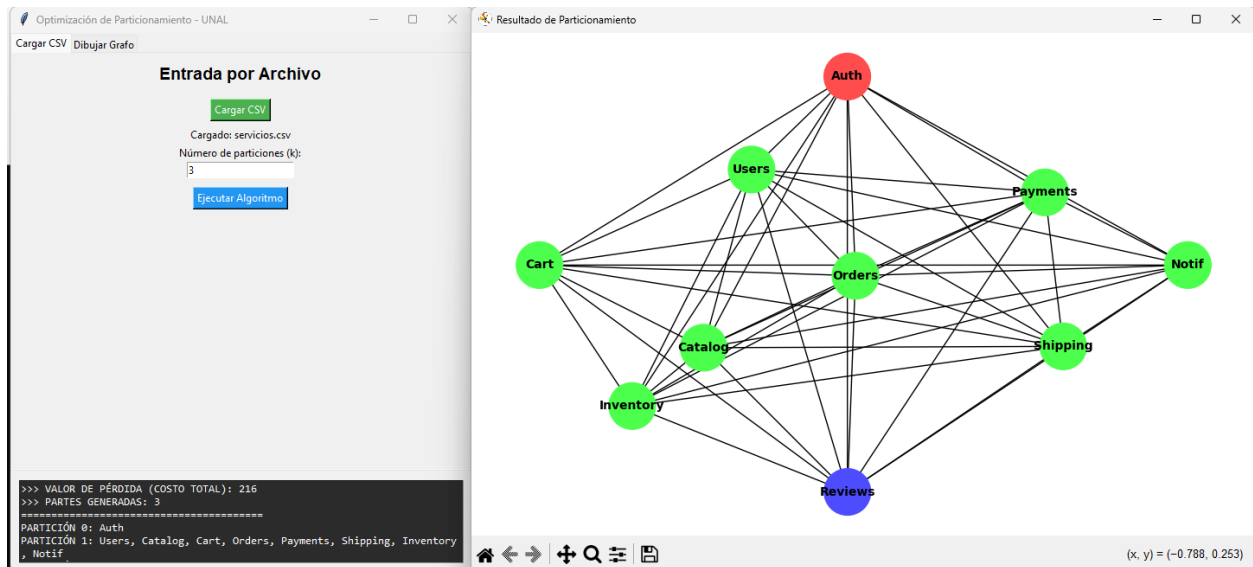


Foto 2. Sistema de Microservicios

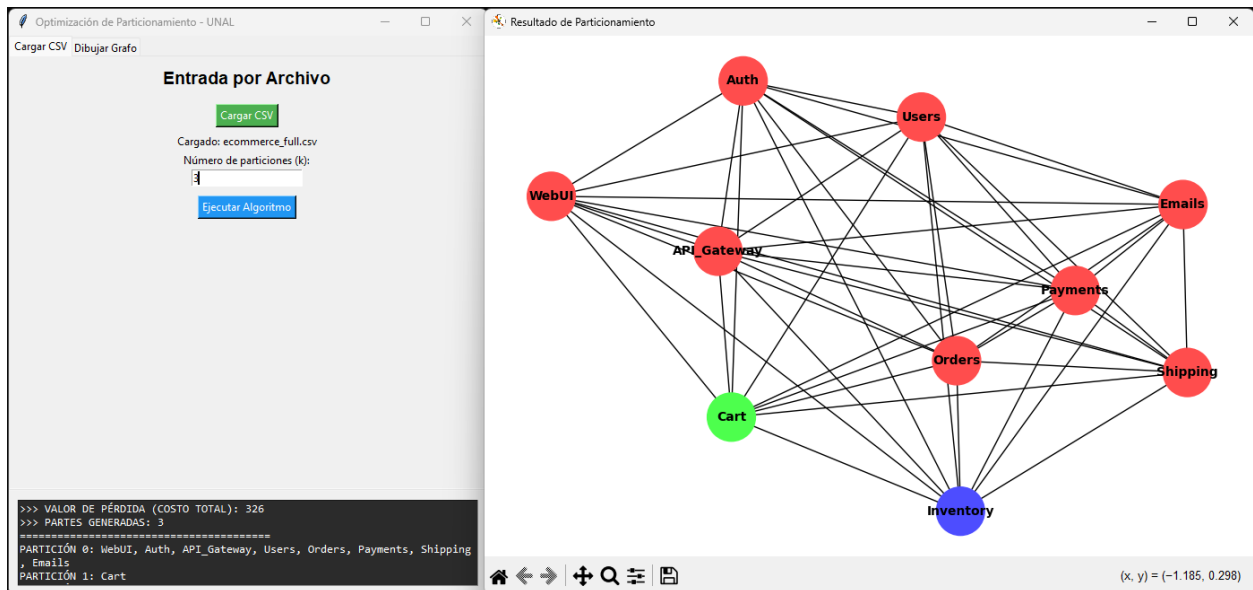


Foto 3. Red Compleja

6. Observaciones Finales

Tras el desarrollo y experimentación con el sistema de optimización, se concluye lo siguiente:

- **Escalabilidad:** La implementación híbrida (Exhaustivo/Heurístico) permite que la herramienta sea útil tanto en el diseño de microservicios pequeños con alta precisión como en el análisis de redes de servicios extensas donde el tiempo de respuesta es crítico.
- **Importancia de la Métrica de Pérdida:** Se determinó que el valor de pérdida no es solo un número, sino un indicador de la cohesión del sistema. Un costo bajo post-

particionamiento refleja una arquitectura más limpia y mantenible.

- **Flexibilidad de Entrada:** La inclusión de un editor gráfico (Canvas) facilita el prototipado rápido de arquitecturas antes de ser documentadas en formatos rígidos como CSV, cumpliendo con las expectativas de una herramienta de ingeniería moderna.
- **Límite Computacional:** Se validó empíricamente que el límite de $N=12$ es el punto de equilibrio óptimo para mantener la interactividad de la interfaz sin sacrificar la exactitud en la mayoría de los casos de uso comunes en administración de sistemas.